UT4 · ACCESO A DATOS

Acceso a bases de datos desde PHP con PDO

Cómo vamos a aprenderlo

El objetivo de esta unidad es aprender a conectar PHP con una base de datos, ejecutar consultas SQL y utilizar los resultados en una página web. No empezaremos construyendo una arquitectura compleja: primero veremos cada operación por separado y añadiremos organización y seguridad cuando ya sepamos qué problema resuelven.

Trabajaremos con PHP, PDO y MySQL/MariaDB. Se presupone que la base de datos y sus tablas pueden crearse previamente desde una herramienta como Workbench o mediante sentencias SQL.

Etapa	Qué aprenderemos
1. Conectar	Crear una conexión PDO y detectar errores.
2. Leer	SELECT, query(), fetch() y fetchAll().
3. Escribir	INSERT, UPDATE, DELETE y exec().
4. Proteger	Consultas preparadas, excepciones y transacciones.
5. Organizar	Reutilizar conexión y separar responsabilidades.

1. Antes de empezar: PHP y una base de datos

Una aplicación web puede necesitar guardar información de forma permanente: usuarios, productos, pedidos, matrículas, etc. Esa información suele almacenarse en una base de datos relacional.

PHP no accede directamente a las tablas como si fueran variables. Envía instrucciones SQL a un sistema gestor de bases de datos (SGBD), que las ejecuta y devuelve un resultado.

1.1. MySQL y MariaDB

- MySQL y MariaDB son sistemas gestores de bases de datos relacionales muy habituales en aplicaciones web.
- Una base de datos contiene tablas; las tablas contienen filas (registros) y columnas (campos).
- InnoDB es el motor de almacenamiento habitual en MySQL/MariaDB y permite, entre otras características, utilizar transacciones e integridad referencial.

En esta unidad utilizaremos MySQL o MariaDB. Los ejemplos SQL son prácticamente iguales para ambos en las operaciones que estudiaremos.

1.2. ¿Qué es PDO?

PDO (PHP Data Objects) es una extensión de PHP que ofrece una interfaz común para acceder a distintos sistemas gestores de bases de datos. Nuestro código PHP utiliza los métodos de PDO y un controlador específico se encarga de comunicarse con MySQL/MariaDB.

Ventaja principal: si en otro proyecto cambiamos de gestor (por ejemplo sqlite), la mayor parte del código PHP puede mantenerse.

1.3. Preparar una tabla de ejemplo

Para que todos los ejemplos sean pequeños y coherentes, utilizaremos una tabla productos:

```
CREATE DATABASE tienda;
```

```
USE tienda;

CREATE TABLE productos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio DECIMAL(8,2) NOT NULL,
    unidades INT NOT NULL
);

INSERT INTO productos (nombre, precio, unidades) VALUES
('Teclado', 25.50, 12),
('Ratón', 14.90, 30),
('Monitor', 179.00, 5);
```

Los ejemplos suponen que el servidor de base de datos está iniciado y que existe un usuario con permisos sobre la base de datos tienda.

2. Primera conexión con PDO

2.1. Los datos que necesita una conexión

Para crear una conexión necesitamos indicar:

- El gestor y los datos del servidor: por ejemplo, MySQL en localhost.
- El nombre de la base de datos.
- El usuario de la base de datos.
- La contraseña de ese usuario.

PDO recibe esta información mediante el DSN (Data Source Name), el nombre de usuario y la contraseña.

2.2. El ejemplo mínimo

```
<?php
$conexion = new PDO(
    'mysql:host=localhost;dbname=tienda;charset=utf8mb4',
    'root',
    ''
);

echo 'Conexión realizada';
?>
```

El DSN empieza por mysql: porque utilizamos el controlador de MySQL. Después indicamos el servidor, la base de datos y la codificación. utf8mb4 permite trabajar correctamente con caracteres Unicode, incluidos muchos símbolos y emojis.

En este primer ejemplo no hemos creado ninguna clase ni archivo adicional. La variable $conexion contiene un objeto PDO que utilizaremos para enviar consultas.

2.3. ¿Qué ocurre si falla la conexión?

La conexión puede fallar, por ejemplo, porque el servidor esté apagado, la base de datos no exista o las credenciales sean incorrectas. PDO puede lanzar una excepción PDOException. Podemos capturarla con try/catch:

```
<?php
try {
    $conexion = new PDO(
        'mysql:host=localhost;dbname=tienda;charset=utf8mb4',
        'root',
        ''
    );

    echo 'Conexión realizada';
} catch (PDOException $e) {
    echo 'No se ha podido conectar con la base de datos';
}
?>
```

Durante el aprendizaje podemos mostrar información técnica para localizar el error. En una aplicación publicada no debemos mostrar al visitante contraseñas, rutas internas ni detalles técnicos de la conexión; se registra el error en un lugar adecuado (un log normalmente) y se muestra un mensaje genérico.

2.4. Configurar el modo de errores

Podemos indicar a PDO que comunique los errores mediante excepciones. Así podremos gestionarlos con try/catch:

```
<?php
$conexion = new PDO(
    'mysql:host=localhost;dbname=tienda;charset=utf8mb4',
    'root',
    '',
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION
    ]
);
?>
```

Esta opción se puede establecer al crear PDO. En los ejemplos siguientes supondremos que la conexión se ha creado con ERRMODE_EXCEPTION.

Práctica breve 1

Crea la base de datos y el archivo conexión.php con el código trabajado. Comprueba que funciona.

3. Consultar datos: SELECT

3.1. Ejecutar una consulta con query()

Cuando queremos recuperar información, utilizamos una sentencia SELECT. El método query() ejecuta la consulta y devuelve un objeto PDOStatement, que representa el resultado de la consulta.

```
<?php
$conexion = new PDO(
    'mysql:host=localhost;dbname=tienda;charset=utf8mb4',
    'root',
    ''
);

$resultado = $conexion->query(
    'SELECT nombre, precio FROM productos'
);
?>
```

Todavía no hemos mostrado los datos. La variable $resultado contiene el conjunto de resultados que debemos recorrer. Podría recorrerse con un foreach, pues PDOStatement es un objeto iterable, pero se suele hacer como muestra el siguiente aparatado porque es más flexible (permite acciones más avanzadas).

3.2. Recuperar una fila con fetch()

El método fetch() obtiene la siguiente fila del resultado. Si ya no quedan filas, devuelve false. Por eso es habitual utilizarlo en un while:

```
<?php
$resultado = $conexion->query(
    'SELECT nombre, precio FROM productos'
);

while ($fila = $resultado->fetch(PDO::FETCH_ASSOC)) {
    echo $fila['nombre'] . ': ';
    echo $fila['precio'] . ' euros<br>';
}
?>
```

PDO::FETCH_ASSOC indica que cada fila se devuelve como un array asociativo: las claves son los nombres de las columnas. Accedemos a los datos mediante $fila['nombre'] y $fila['precio'].

3.3. Recuperar todas las filas con fetchAll()

Si el resultado es pequeño, podemos recuperar todas las filas de una vez mediante fetchAll():

```
<?php
$resultado = $conexion->query(
    'SELECT nombre, precio FROM productos'
);
```

```
$productos = $resultado->fetchAll(PDO::FETCH_ASSOC);

foreach ($productos as $producto) {
    echo $producto['nombre'] . '<br>';
}
?>
```

Diferencia importante:

- fetch(): obtiene una fila cada vez. Es adecuado para recorrer resultados grandes sin cargar todas las filas simultáneamente.
- fetchAll(): obtiene todas las filas y las guarda en un array. Es cómodo para resultados pequeños o moderados, pero puede consumir mucha memoria si hay miles de registros. Por el contrario, es la mejor opción para recorrer el array varias veces con posterioridad, o convertir el array en JSON, ...

3.4. Consultas con condiciones

SQL permite filtrar, ordenar y limitar los resultados. Por ejemplo:

```
<?php
$resultado = $conexion->query(
    'SELECT nombre, precio
     FROM productos
     WHERE unidades > 10
     ORDER BY precio DESC'
);

while ($fila = $resultado->fetch(PDO::FETCH_ASSOC)) {
    echo $fila['nombre'] . ' - ' . $fila['precio'] . '<br>';
}
?>
```

En este ejemplo todos los valores de la consulta están escritos en el propio código. Más adelante veremos qué hacer cuando una condición depende de lo que introduce el usuario.

Práctica breve 2

Muestra todos los productos con su precio y unidades. Después muestra únicamente los productos que tengan menos de 10 unidades, ordenados por precio de menor a mayor.

4. Modificar datos: INSERT, UPDATE y DELETE

No todas las sentencias SQL devuelven filas. INSERT, UPDATE y DELETE modifican la información almacenada. Para estas operaciones sencillas podemos utilizar exec(), que devuelve el número de filas afectadas.

4.1. Insertar un registro con exec()

```
<?php
$filas = $conexion->exec(
```

```
    "INSERT INTO productos (nombre, precio, unidades)
     VALUES ('Altavoces', 32.90, 8)"
);

echo "Filas insertadas: " . $filas;
?>
```

Si la inserción se realiza correctamente, normalmente se verá afectada una fila. exec() no devuelve el identificador generado automáticamente. En caso de error lanzará una excepción según configuramos la conexión al principio (si no hubiéramos hecho ese paso, devolvería false).

4.2. Obtener el último identificador

```
<?php
$conexion->exec(
    "INSERT INTO productos (nombre, precio, unidades)
     VALUES ('Webcam', 45.00, 6)"
);

$idNuevo = $conexion->lastInsertId();
echo "Nuevo producto: " . $idNuevo;
?>
```

lastInsertId() devuelve el último identificador generado por una columna AUTO_INCREMENT en esa conexión. Es decir, un exec con un INSERT hace 2 cosas: devuelve un entero con el número de inserciones realizadas y actualiza el objeto conexión, añadiendo el último id insertado.

4.3. Actualizar registros

```
<?php
$filas = $conexion->exec(
    "UPDATE productos
     SET precio = 29.90
     WHERE id = 1"
);

echo "Filas modificadas: " . $filas;
?>
```

4.4. Eliminar registros

```
<?php
$filas = $conexion->exec(
    "DELETE FROM productos
     WHERE id = 3"
);
```

```
echo "Filas eliminadas: " . $filas;
?>
```

¡Atención! Un UPDATE o DELETE sin WHERE puede afectar a todos los registros de la tabla. Antes de ejecutar estas operaciones hay que comprobar cuidadosamente la condición.

Resumen inicial: query() se utiliza habitualmente para SELECT; exec() para sentencias que no devuelven un conjunto de filas. En ambos casos estamos usando consultas escritas directamente en el código. Para datos externos utilizaremos consultas preparadas.

Práctica breve 3

Inserta un producto, muestra su ID, modifica su precio y finalmente elimina ese mismo producto. Comprueba el número de filas afectadas en cada operación.

5. Consultas preparadas: seguridad y datos variables

5.1. El problema de concatenar datos

Imagina que queremos buscar un producto cuyo nombre introduce el usuario. Una solución tentadora sería concatenar ese texto dentro del SQL:

```
<?php
$nombre = $_GET['nombre'];

$sql = "SELECT * FROM productos WHERE nombre = '$nombre'";
$resultado = $conexion->query($sql);
?>
```

Esta forma de construir consultas es peligrosa. Si el usuario introduce código SQL, podría alterar la consulta. Este problema se conoce como inyección SQL.

5.2. prepare() y execute()

La solución habitual es utilizar una consulta preparada. Primero escribimos la consulta con marcadores para los valores que cambiarán. Después enviamos los valores por separado.

```
<?php
$sql = 'SELECT * FROM productos WHERE nombre = :nombre';

$consulta = $conexion->prepare($sql);

$consulta->execute([
    ':nombre' => $_GET['nombre']
]);

$producto = $consulta->fetch(PDO::FETCH_ASSOC);
?>
```

Los dos pasos principales son:

1. prepare(): prepara la sentencia SQL y devuelve un objeto PDOStatement.
2. execute(): ejecuta la sentencia proporcionando los valores de los parámetros.

Los parámetros separan los datos introducidos por el usuario de las instrucciones SQL. Es la forma que debemos utilizar cuando una consulta incorpora valores externos.

5.3. Marcadores con nombre y marcadores posicionales

Podemos identificar los parámetros de dos maneras. Para empezar, recomendamos los marcadores con nombre porque facilitan la lectura.

```
// Marcadores con nombre
$sql = 'SELECT * FROM productos WHERE precio < :limite';
$consulta = $conexion->prepare($sql);
$consulta->execute([':limite' => 50]);

// Marcadores posicionales
$sql = 'SELECT * FROM productos WHERE precio < ?';
$consulta = $conexion->prepare($sql);
$consulta->execute([50]);
```

El orden de los valores del array debe corresponderse con el orden de los marcadores cuando utilizamos ?. Con marcadores con nombre, cada valor se asocia a su marcador.

5.4. Consultas preparadas para INSERT, UPDATE y DELETE

```
<?php
// INSERT
$sql = 'INSERT INTO productos (nombre, precio, unidades)
        VALUES (:nombre, :precio, :unidades)';
$consulta = $conexion->prepare($sql);
$consulta->execute([
    ':nombre' => 'Auriculares',
    ':precio' => 39.90,
    ':unidades' => 10
]);

// UPDATE
$sql = 'UPDATE productos SET precio = :precio WHERE id = :id';
$consulta = $conexion->prepare($sql);
$consulta->execute([
    ':precio' => 34.90,
    ':id' => 1
]);

// DELETE
$sql = 'DELETE FROM productos WHERE id = :id';
```

```
$consulta = $conexion->prepare($sql);
$consulta->execute([':id' => 2]);
?>
```

5.5. Qué se puede parametrizar

Los parámetros representan valores, no fragmentos de SQL. Podemos parametrizar el valor de una condición, pero no sustituir directamente con un parámetro el nombre de una tabla, el nombre de una columna o la palabra ASC/DESC.

Práctica breve 4

Crea un buscador que reciba un precio máximo y muestre los productos cuyo precio sea inferior o igual a esa cantidad. Utiliza prepare() y execute().

6. Gestionar errores y transacciones

6.1. Capturar errores de una consulta

Si hemos configurado PDO con PDO::ERRMODE_EXCEPTION, los errores de conexión o de ejecución pueden generar excepciones. Podemos controlar el error sin mostrar información sensible al usuario:

```
<?php
try {
    $consulta = $conexion->prepare(
        'SELECT * FROM productos WHERE id = :id'
    );
    $consulta->execute([':id' => 1]);

    $producto = $consulta->fetch(PDO::FETCH_ASSOC);
} catch (PDOException $e) {
    error_log($e->getMessage());
    echo 'No ha sido posible realizar la operación.';
}
?>
```

error_log() permite enviar el detalle técnico al registro de errores configurado en PHP. En una práctica local se puede consultar el mensaje técnico para aprender a localizar el problema, pero no conviene mostrarlo directamente en una web pública.

6.2. Qué es una transacción

Una transacción agrupa varias operaciones para que se confirmen todas juntas o se deshagan si algo falla. Es útil cuando varias modificaciones forman una única operación lógica.

Por ejemplo, al transferir dinero entre dos cuentas hay que restar de una y sumar en otra. No queremos que se reste el dinero si después falla el ingreso en la otra cuenta.

6.3. Los tres métodos principales

- beginTransaction(): inicia la transacción.

- commit(): confirma los cambios.
- rollBack(): deshace los cambios de la transacción.

```php
<?php
try {
    $conexion->beginTransaction();

    $conexion->exec(
        'UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1'
    );

    $conexion->exec(
        'UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2'
    );

    $conexion->commit();
    echo 'Operación realizada';
} catch (PDOException $e) {
    if ($conexion->inTransaction()) {
        $conexion->rollBack();
    }

    echo 'No se pudo completar la operación';
}
?>
```

El ejemplo ilustra el mecanismo. El inTransaction en el if cubre el caso en el que beginTransaction de error. En ese caso, intentar hacer rollBack generaría otra excepción (al no estar en una transacción, pues falló antes de iniciarla).

En una aplicación real también habría que comprobar que las cuentas existen, que la primera dispone de saldo suficiente y que las cantidades son válidas.

Práctica breve 5

Simula una operación que requiera dos UPDATE. Provoca deliberadamente un error en la segunda operación y comprueba que, al ejecutarse rollBack(), tampoco queda aplicado el primer cambio.

7. Organizar el código sin complicarlo

Hasta ahora hemos escrito la conexión y las consultas en un mismo archivo. Es adecuado para aprender, pero repetir la conexión en cada página resulta incómodo. Vamos a mejorar la organización en pasos pequeños.

7.1. Primer paso: mover la conexión a un archivo

Creamos conexion.php. Su única responsabilidad será crear y devolver la conexión PDO:

```php
<?php
```

```
try {
    $conexion = new PDO(
        'mysql:host=localhost;dbname=tienda;charset=utf8mb4',
        'root',
        '',
        [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
        ]
    );
} catch (PDOException $e) {
    error_log($e->getMessage());
    die('No se ha podido conectar con la base de datos');
}
?>
```

En otro archivo incluimos la conexión:

```
<?php
require_once 'conexion.php';

$resultado = $conexion->query('SELECT * FROM productos');

while ($producto = $resultado->fetch()) {
    echo $producto['nombre'] . '<br>';
}
?>
```

require_once incluye el archivo una sola vez durante la ejecución del script. Así evitamos repetir el código de conexión en todas las páginas.

7.2. Segundo paso: separar la configuración

Las credenciales no deberían estar dispersas por el código. En un primer nivel didáctico, podemos crear un archivo configuracion.php que contenga los datos de conexión y que no se publique como contenido accesible desde el navegador.

```
<?php
// configuracion.php
$host = 'localhost';
$baseDatos = 'tienda';
$usuario = 'root';
$contrasena = '';
?>
<?php
// conexion.php
require_once 'configuracion.php';
```

```
try {
    $dsn = "mysql:host=$host;dbname=$baseDatos;charset=utf8mb4";
    $conexion = new PDO($dsn, $usuario, $contrasena, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
    ]);
} catch (PDOException $e) {
    error_log($e->getMessage());
    die('Error de conexión');
}
?>
```

Separar el archivo de configuración no cifra ni protege por sí solo las credenciales. Hay que evitar que el archivo se descargue o se exponga, limitar los permisos del usuario de base de datos y no subir credenciales reales a repositorios públicos.

7.3. Variables de entorno y archivos .env: ampliación

En proyectos reales es frecuente mantener las credenciales fuera del código fuente mediante variables de entorno. Un archivo .env es una forma cómoda de definirlas durante el desarrollo, pero PHP no interpreta automáticamente ese archivo.

Para cargarlo con una librería como phpdotenv normalmente se utiliza Composer. Esto introduce herramientas y dependencias externas que no son necesarias para aprender PDO. Por eso lo dejamos como ampliación, una vez comprendido el proceso básico de conexión.

```
# Ejemplo de archivo .env (no es código PHP)
DB_HOST=localhost
DB_NAME=tienda
DB_USER=usuario_app
DB_PASSWORD=una_clave
```

El archivo .env debe excluirse del control de versiones cuando contiene secretos. En producción es preferible utilizar el sistema de variables de entorno o de secretos del propio entorno de despliegue.

8. Separar las consultas de la página: introducción al modelo

Cuando una aplicación crece, conviene que el archivo que presenta HTML no contenga todas las consultas SQL. Podemos agrupar las operaciones relacionadas con productos en un archivo independiente. A ese componente lo llamaremos modelo.

En este primer acercamiento no necesitamos clases ni patrones de diseño. Basta con funciones que reciben la conexión como parámetro.

8.1. Archivo producto_modelo.php

```
<?php
function obtenerProductos(PDO $conexion): array {
    $consulta = $conexion->query(
```

```
        'SELECT id, nombre, precio, unidades
         FROM productos
         ORDER BY nombre'
    );

    return $consulta->fetchAll(PDO::FETCH_ASSOC);
}

function obtenerProductoPorId(PDO $conexion, int $id): array|false {
    $consulta = $conexion->prepare(
        'SELECT id, nombre, precio, unidades
         FROM productos
         WHERE id = :id'
    );

    $consulta->execute([':id' => $id]);

    return $consulta->fetch(PDO::FETCH_ASSOC);
}
?>
```

8.2. Archivo index.php

```
<?php
require_once 'conexion.php';
require_once 'producto_modelo.php';

$productos = obtenerProductos($conexion);
?>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Productos</title>
</head>
<body>
    <h1>Listado de productos</h1>

    <?php foreach ($productos as $producto): ?>
        <p>
            <?= htmlspecialchars($producto['nombre']) ?>
            - <?= htmlspecialchars((string) $producto['precio']) ?> €
```

```
        </p>
    <?php endforeach; ?>
</body>
</html>
```

Ahora cada archivo tiene una responsabilidad más clara: conexion.php crea la conexión, producto_modelo.php contiene operaciones de acceso a datos e index.php coordina la página y presenta los resultados.

Observa también el uso de htmlspecialchars() al insertar datos en el HTML. Las consultas preparadas protegen frente a la inyección SQL, pero no sustituyen el escape adecuado de los datos al mostrarlos en una página web.

8.3. ¿Y la inyección de dependencias?

En el ejemplo anterior, las funciones reciben la conexión PDO como parámetro. Eso ya permite que la conexión se cree en un lugar y se entregue a las funciones que la necesitan. A esta idea general se la conoce como inyección de dependencias: un componente recibe aquello que necesita en lugar de crearlo por su cuenta.

8.4. CRUD completo

CRUD son las siglas de Create, Read, Update y Delete. Una vez entendidos por separado INSERT, SELECT, UPDATE y DELETE, se pueden reunir en un modelo de productos con cuatro operaciones. Todas las operaciones que reciben valores externos deben utilizar consultas preparadas.

Operación	SQL	Método PDO habitual
Create	INSERT	prepare() + execute()
Read	SELECT	query() o prepare() + execute()
Update	UPDATE	prepare() + execute()
Delete	DELETE	prepare() + execute()

Práctica breve 6

Completar el CRUD, llevando las funciones de insertar y eliminar al archivo producto_modelo.php.

9. Resumen de métodos y conceptos

Elemento	Para qué sirve	Cuándo aparece
new PDO()	Crear una conexión.	Al iniciar el acceso a datos.
query()	Ejecutar una consulta que devuelve un resultado.	SELECT sencillo sin valores externos.
exec()	Ejecutar una sentencia y obtener filas afectadas.	INSERT/UPDATE/DELETE sencillos.
prepare()	Preparar una sentencia con parámetros.	Cuando la consulta incorpora valores variables.
execute()	Ejecutar la sentencia preparada.	Después de prepare().
fetch()	Obtener la siguiente fila.	Recorrer resultados fila a fila.
fetchAll()	Obtener todas las filas en un array.	Resultados pequeños o moderados.

lastInsertId()	Obtener el último ID autogenerado.	Después de un INSERT compatible.
beginTransaction()	Iniciar una transacción.	Agrupar varias operaciones.
commit()	Confirmar una transacción.	Cuando todas las operaciones han ido bien.
rollBack()	Deshacer una transacción.	Cuando alguna operación falla.
require_once	Incluir un archivo una sola vez.	Reutilizar conexión o funciones.

11. Práctica final

Integración final: crear una pequeña aplicación CRUD de productos, integrando los contenidos de este pdf con los del RA anterior (gestión de formularios en el servidor).